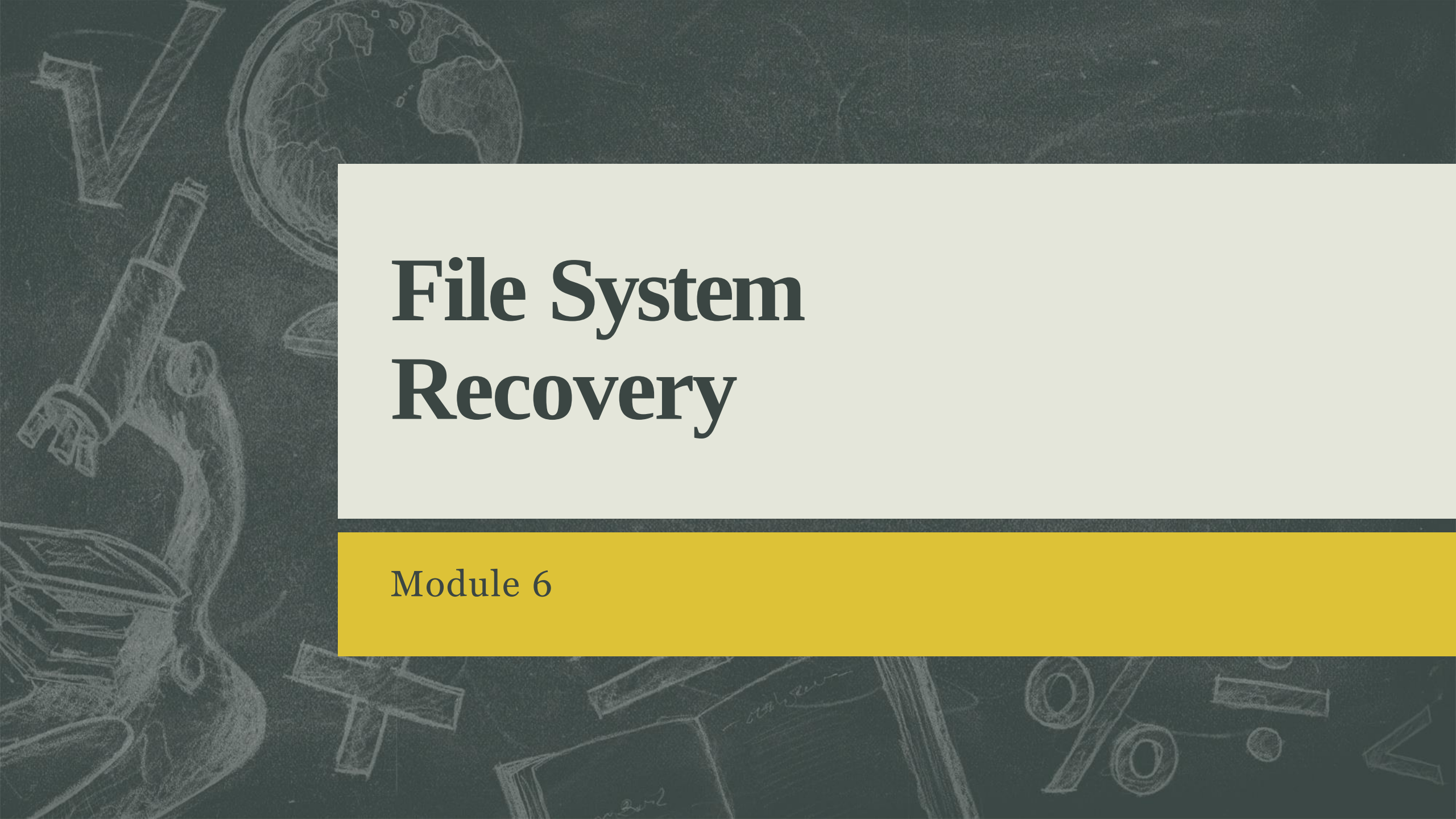




File System Recovery

Module 6

File System Recovery

- The process of restoring access to files and directories after they've been corrupted, deleted, or made inaccessible due to system failures, software issues, or user errors
- The goal is to retrieve as much data as possible and return the file system to a stable, usable state.

File System Recovery

- **File System Corruption:** This happens when metadata (information about files, directories, and file structure) is damaged, often due to power failures, system crashes, or abrupt shutdowns.
- **Data Consistency and Integrity:** Recovery aims to ensure that the file system's structure and the actual data stored within it remain consistent. Techniques like journaling help maintain this consistency.
- **File Metadata:** Information such as file size, location, permissions, and modification timestamps. Metadata often plays a crucial role in recovery.

File System Recovery - Backups and Snapshots

- The simplest method is restoring from a recent **backup or snapshot**, which involves storing periodic copies of files.
- This method can quickly recover lost data but may lose changes made after the last backup.
- *Backups are copies of files, directories, or entire disk images* stored separately from the main file system.
- In the event of data loss or corruption, backups provide a way to restore data to a known, stable state.

File System Recovery - Backups and Snapshots

- **File-Level Backup:** Allows recovery of individual files, minimizing the impact on the rest of the system.
- **Disk Image Backup:** Backs up the entire disk image, including the operating system, applications, and configurations.
 - This is especially helpful when recovering from a catastrophic failure, as the entire system state can be restored.

File System Recovery - Backups and Snapshots

- **Identify the Restore Point:** Determine the most recent backup that doesn't include the corrupted or missing data.
- **Restore Data:** Depending on the backup type, you can restore individual files, directories, or the entire system.
- **Verify Integrity:** Check the restored files or system state to ensure they're consistent and usable

File System Recovery - Journaling

- A **journal** is a data structure that records changes to a file system before they are committed to the main part of the file system
- A journaling file system **records changes to files and metadata** in a dedicated area **called the journal** before **applying them to the main file system**.
- This approach **improves data integrity and speeds up recovery** after unexpected shutdowns, system crashes, or power failures.

File System Recovery – Journaling - Working

- **Transactional Logging:** When a file system operation (e.g., creating, deleting, or modifying a file) is requested, the **journaling system records the operation in the journal before applying it to the main data area.**
- **Two-Phase Write Process:**
 - **Journal Write:** The file system writes an entry (transaction) in the journal, detailing what changes are about to be made.
 - **Main Write:** Once the transaction is fully written to the journal, the file system proceeds with the actual changes to the files and metadata.

File System Recovery – Journaling - Working

- **Checkpointing:** The system periodically flushes (or "commits") transactions from the journal to the main file system, creating checkpoints that maintain consistency.
- **Automatic Rollback/Roll-Forward:** In the event of a crash, the file system can use the journal to:
 - Rollback: Discard incomplete or corrupted transactions, preserving a stable state.
 - Roll Forward: Complete partially applied transactions using the information in the journal, restoring the system to the last known good state.
- **Reduced File System Check Times:** Non-journaling file systems, like ext2, require lengthy checks to identify inconsistencies after a crash. Journaling file systems reduce recovery time, as only the journal needs to be processed, not the entire file system.
- **Enhanced Data Consistency:** Journaling ensures that operations are atomic, meaning they either complete fully or not at all, which is critical for preventing partial data corruption.

File System Recovery - Soft Updates

- Soft updates is a technique used in file systems to **maintain data consistency without** the need for a **full journaling approach**.
- Introduced to **address the performance costs** associated with traditional journaling
- Soft updates ensure that **file system metadata is written to disk in a specific, carefully managed order** to prevent corruption during unexpected shutdowns, crashes, or power losses.

File System Recovery - Soft Updates

- **Metadata Dependency Management:** Soft updates **enforce strict ordering rules on metadata updates to ensure that dependencies** between operations are maintained.
 - For example, if a file is being moved, the system first updates the destination directory to include the file before removing it from the source directory.
- **In-Memory Metadata Tracking:** Instead of writing metadata changes immediately to disk, soft updates hold these changes in memory and only write them to disk, when necessary, in the correct order, to avoid inconsistency.
- **Optimized Disk I/O:** By consolidating related updates and reducing redundant writes, soft updates aim to **minimize disk I/O**, which leads to better performance compared to conventional journaling file systems.

File System Recovery - Soft Updates

- **Deferred Metadata Writes:**
 - When changes occur, such as creating or deleting files, the metadata is modified in memory.
 - Instead of immediately flushing each change to disk, soft updates delay these writes until they can be written in a batch, which reduces the number of disk operations.
- **Ordering Constraints:**
 - Soft updates implement rules that define the sequence of writes to avoid leaving the file system in an inconsistent state. For instance:
 - *Directory entries are created only after the inode (file metadata structure) is initialized.*
 - *An inode is removed only after the directory entry pointing to it is deleted.*
 - *Blocks are deallocated only after all references to them are removed.*

File System Recovery – Log Structured File System

- A Log-Structured File System (LFS) is a type of file system that writes all changes (both data and metadata) sequentially in the form of a log, rather than directly modifying specific blocks on disk.
- This design is optimized for systems where write performance is crucial, such as in workloads with high write rates or on storage with high latency for random writes, like magnetic disks or flash storage.

File System Recovery – Log Structured File System

- When a write operation occurs, LFS gathers data and metadata changes into a log segment.
- The file system writes this segment to the end of the log rather than directly modifying any existing data blocks on disk.
 - New Data Blocks: New data is appended to the end of the log.
 - Updated Data Blocks: When files are updated, the new data is appended to the log rather than overwriting the original blocks. This results in multiple versions of data blocks for the same file, with the latest version considered the current one.

File System Recovery – Log Structured File System

- Indexing for Fast Access:
 - Because the file system is appended sequentially, finding specific data requires an index.
 - LFS maintains an in-memory data structure, like an inode map, to keep track of the latest location of each file's data blocks in the log.

File System Recovery – Log Structured File System

Garbage Collection:

- As data is updated, old versions of data blocks accumulate in the log, taking up space. To manage this, LFS periodically runs garbage collection to identify and free up space taken by outdated data blocks.
- **Segment Cleaning:** The file system divides the log into segments and examines each to see if it contains any "live" data (data that's still in use). If a segment contains old versions of data or files that have been deleted, the segment can be marked for reuse.
- **Efficiency:** Cleaning can impact performance, so algorithms are often used to prioritize which segments to clean, aiming to maximize space reclamation while minimizing impact on the system.

File System Recovery – Log Structured File System

- Crash Recovery:
 - If a crash occurs, LFS uses the most recent log entries to reconstruct the file system's state.
 - By replaying these entries, the file system can quickly be brought to a consistent state, without lengthy checks of the entire disk.

Network File System

- a remote directory is mounted over (and cover) a local file system directory
mounting operation is not transparent
- the host name of the remote directory has to be provided designed for heterogeneous environment with the help of RPC
- different machine architecture, OS, or network architecture to improve performance, NFS employs many caches
- directory name cache, file block cache, file attribute cache...

Network File System

- A **server exports** (shares) **directories and files**, and **clients mount these directories** to access files as if they were part of the local filesystem.
- To access files, the client "mounts" the remote directory using the NFS mount protocol.
- This process connects the client's file system to the server's shared directory, making files accessible to the client system as if they are local files.
- NFS is designed to be stateless, meaning that the server doesn't maintain session information about connected clients. This makes it resilient against network interruptions and simplifies recovery if connections are lost.
- NFS uses RPC to enable file operations over the network. When a client wants to access a file on the server, it sends an RPC request to perform actions like reading, writing, or creating files. The server responds with the results.

Network File System. - NFS Mount Protocol

- The NFS mount protocol is the process by which a client connects to a server's shared directory. The mount protocol occurs before file access and involves the following steps:
 - The server specifies which directories are available to share and which clients can access them. This is configured in the server's exports file (often /etc/exports on Linux), where each exported directory and its access permissions are defined.
 - Client Requests a Mount: The client initiates a mount request to the NFS server using the server's IP address and the directory path. This request is handled by the server's mount daemon (mountd).
 - Authentication: The server checks if the client has permission to access the requested directory. Permissions are typically based on the client's IP address and user credentials (such as UID and GID).
 - Server Sends File Handle: If the client is authorized, the server returns a file handle. This file handle is a unique identifier representing the requested directory, allowing the client to reference it in future requests.
 - Client Mounts the Directory: The client's OS mounts the directory using the file handle provided by the server, making it available on the client as a directory within the local filesystem.